

QUEST INNOVATIVE SOLUTIONS

Python Full Stack Development — Python Core Exam (Session 1)

Answer Key | Max Marks: 50 | Duration: 2 Hours

PART A — Short Answers (2 marks each)

Q1. Differentiate between the 'is' operator and the '==' operator.

'==' checks whether two variables have the **same value** (equality). 'is' checks whether two variables point to the **same object in memory** (identity). *Example:* `a = [1, 2, 3]` `b = [1, 2, 3]` `a == b` → True (same values) `a is b` → False (different objects in memory)

Q2. Differentiate between the 'for' loop and the 'while' loop.

for loop: Used when the number of iterations is *known*. It iterates over a sequence (list, range, string, etc.). **while loop:** Used when the number of iterations is *unknown* and depends on a condition being True.
`for i in range(5): ...` ← runs exactly 5 times `while x > 0: ...` ← runs as long as `x > 0`

Q3. What are operators in Python?

Operators are special symbols that perform operations on variables and values. Types: Arithmetic (+, -, *, /), Comparison (==, !=, >, <), Logical (and, or, not), Assignment (=, +=), Bitwise (&, |, ^), Membership (in, not in), Identity (is, is not).

Q4. What is the replace() method in Python strings?

replace(old, new[, count]) returns a new string where all (or a specified number of) occurrences of a substring are replaced with another substring. `s = "hello world"` `s.replace("world", "Python")` → "hello Python" The original string is unchanged (strings are immutable).

Q5. Differentiate between mutable and immutable objects. Give two examples each.

Immutable — cannot be changed after creation. Examples: **int**, **str**, **tuple** **Mutable** — can be modified after creation. Examples: **list**, **dict**, **set** `x = "hello"; x[0] = 'H'` → TypeError (immutable) `lst = [1,2,3]; lst[0] = 9` → works fine (mutable)

Q6. Differences between a list and a tuple (at least three points each).

List: • Mutable — elements can be added, removed, or changed. • Defined with square brackets: `[1, 2, 3]` • Slower than tuple due to dynamic nature. • Use case: collection of items that may change (e.g., shopping cart). **Tuple:** • Immutable — cannot be changed after creation. • Defined with parentheses: `(1, 2, 3)` • Faster and uses less memory. • Use case: fixed data like coordinates, database records.

Q7. How can you create a tuple containing only a single number 10?

Add a trailing comma inside the parentheses: `t = (10,)` ← correct, type is tuple `t = (10)` ← wrong, this is just an int The comma is what makes Python treat it as a tuple.

Q8. How can you create an empty dictionary and an empty set?

Empty dictionary: `d = {}` or `d = dict()` **Empty set:** `s = set()` ← must use `set()`, because `{}` creates a dict, not a set.

Q9. Define local variable and global variable. State one key difference.

Local variable: Declared inside a function; accessible only within that function. **Global variable:** Declared outside all functions; accessible throughout the program. **Key difference:** A local variable ceases to exist when the function returns; a global variable persists for the entire program lifetime.

Q10. Main difference between pass by value and pass by reference.

Pass by value: A copy of the variable is passed. Changes inside the function do not affect the original variable. **Pass by reference:** The reference (address) of the variable is passed. Changes inside the function affect the original. Python uses *pass by object reference* — immutable objects behave like pass-by-value, mutable objects behave like pass-by-reference.

Q11. One key difference between *args and **kwargs.

args** collects extra *positional* arguments into a tuple. *kwargs** collects extra *keyword* arguments into a dictionary. `def f(*args): ... ← f(1, 2, 3)` `def f(**kwargs): ... ← f(a=1, b=2)`

Q12. What is recursion in Python?

Recursion is when a function calls **itself** to solve a smaller sub-problem of the same type. Every recursive function must have a **base case** to stop the recursion. `def factorial(n): if n == 0: return 1 ← base case` `return n * factorial(n-1)`

Q13. What is polymorphism in Python?

Polymorphism means "many forms". It allows the same method/function name to behave differently based on the object or arguments. In Python it is achieved through method overriding and duck typing. `class Dog: def sound(self): return "Woof"` `class Cat: def sound(self): return "Meow"` Both have `sound()`; the same call works on different objects.

Q14. What is encapsulation in Python?

Encapsulation is the OOP concept of **bundling data (attributes) and methods** inside a class, and restricting direct access to some of them. Achieved using: • Single underscore `_var` — conventionally private (not enforced) • Double underscore `__var` — name-mangled (strongly private) It protects data integrity and hides internal implementation.

Q15. What is a constructor and a destructor? Explain the difference.

Constructor (`__init__`): Automatically called when an object is *created*. Used to initialise object attributes.

Destructor (`__del__`): Automatically called when an object is *deleted* or garbage collected. Used to release resources. **Key difference:** Constructor initialises the object at creation; destructor cleans up when the object is destroyed.

Q16. What is a lambda function? State its syntax.

A lambda function is an **anonymous, single-expression function** defined in one line using the **lambda** keyword. Syntax: **lambda arguments : expression** Example: `square = lambda x: x ** 2` `square(5) → 25` Used commonly with `map()`, `filter()`, `sorted()`.

Q17. What are file modes in Python?

Common file modes used with `open()`: • **'r'** — read (default); error if file doesn't exist • **'w'** — write; creates or overwrites file • **'a'** — append; adds to end of file • **'x'** — create; error if file exists • **'b'** — binary mode (e.g., `'rb'`, `'wb'`) • **'+'** — read and write (e.g., `'r+'`, `'w+'`)

Q18. How can you delete a file in Python?

Use the **os** module: `import os os.remove("filename.txt")` Always check if the file exists first to avoid errors: `if os.path.exists("filename.txt"): os.remove("filename.txt")`

Q19. What is a class and an object in Python?

Class: A blueprint/template that defines attributes and methods. Created using the **class** keyword.

Object: An instance (real-world entity) created from a class. `class Car: ← class` `def __init__(self, name): self.name = name my_car = Car("Tesla") ← object`

Q20. One key difference between a library, a module, and a package.

Module: A single .py file containing Python code (functions, classes, variables). **Package:** A directory of multiple modules with an `__init__.py` file. **Library:** A broader collection of packages and modules (can include C extensions); e.g., NumPy, requests. In short: `module ⊂ package ⊂ library`.

PART B — Long Answers (5 marks each)

Q21. Explain inheritance in Python and describe its different types with examples.

Inheritance is an OOP feature that allows a **child class** to inherit attributes and methods from a **parent class**, promoting code reusability. Syntax: **class Child(Parent):** **Types of Inheritance:**

1. Single Inheritance: One child inherits from one parent.

```
class Animal:
    def speak(self): print('Sound')
class Dog(Animal):
    def bark(self): print('Woof')
d = Dog()
d.speak() # inherited
```

2. Multiple Inheritance: One child inherits from more than one parent.

```
class A:
    def method_a(self): print('A')
class B:
    def method_b(self): print('B')
class C(A, B): pass
c = C()
c.method_a(); c.method_b()
```

3. Multilevel Inheritance: A chain: grandparent → parent → child.

```
class Animal:
    def breathe(self): print('Breathing')
class Mammal(Animal):
    def warm_blooded(self): print('Warm blooded')
class Dog(Mammal):
    def bark(self): print('Woof')
d = Dog()
d.breathe() # from Animal
```

4. Hierarchical Inheritance: Multiple children inherit from one parent.

```
class Vehicle:
    def start(self): print('Starting')
class Car(Vehicle): pass
class Bike(Vehicle): pass
Car().start(); Bike().start()
```

5. Hybrid Inheritance: Combination of two or more types of inheritance.

```
# Combination of multiple and multilevel
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass # MRO resolves order
```

The **super()** function is used to call the parent class constructor or methods from within the child class.

Q22. Explain exception handling in Python in detail.

What is an Exception?

An exception is an error that occurs during the **execution** of a program, disrupting the normal flow.

Examples: ZeroDivisionError, FileNotFoundError, TypeError, ValueError, IndexError.

Purpose of Exception Handling:

It allows a program to handle runtime errors **gracefully** without crashing. This improves reliability, provides meaningful error messages, and allows recovery or cleanup.

Basic Syntax (try-except-else-finally):

```
try:
    # code that may raise an error
except ExceptionType:
    # handle the error
else:
    # runs if no exception occurred
finally:
    # always runs (cleanup code)
```

Handling Multiple Exceptions:

```
try:
    x = int(input('Enter number: '))
    result = 10 / x
except ValueError:
    print('Please enter a valid integer')
except ZeroDivisionError:
    print('Cannot divide by zero')
except Exception as e:
    print(f'Unexpected error: {e}')
else:
    print(f'Result: {result}')
finally:
    print('Execution complete')
```

Example Scenario — File Reading:

```
try:
    with open('data.txt', 'r') as f:
        content = f.read()
except FileNotFoundError:
    print('File not found. Please check the path.')
except PermissionError:
    print('You do not have permission to read this file.')
finally:
    print('File operation attempted.')
```

Without exception handling, the above code would crash with a traceback. With handling, the user gets a friendly message and the program continues or exits cleanly. The **finally** block is ideal for closing database connections, releasing file handles, etc.

— End of Answer Key —